

Kalman Filter and application on Ball-Plate system

By

Mahendra Singh

Rohan Upadhye

Christoph Mailhammer

30 April 2026

Course: Kalman Filtering in Control and Communication Applications

Instructors: Prof.Dr. Birger Mysliwetz , Prof.Dr. Markus Stichler

Introduction

- **Goal:** Stabilize ball on plate
- **Control input:** Servo angle
- **Measurement:** Ball position (camera)

! Problem:

- Velocity not measurable
- other Internal states(Plate angle)

State Estimation

- **What?**
hidden states(velocity, angle) measurement
- **Why?**
to better monitor plant
to reduce sensor cost
to use the estimated states in 'State Feedback Controller'
- **How?**
Using measured output
Using the mathematical model of system
Using the present noise in model and measurement

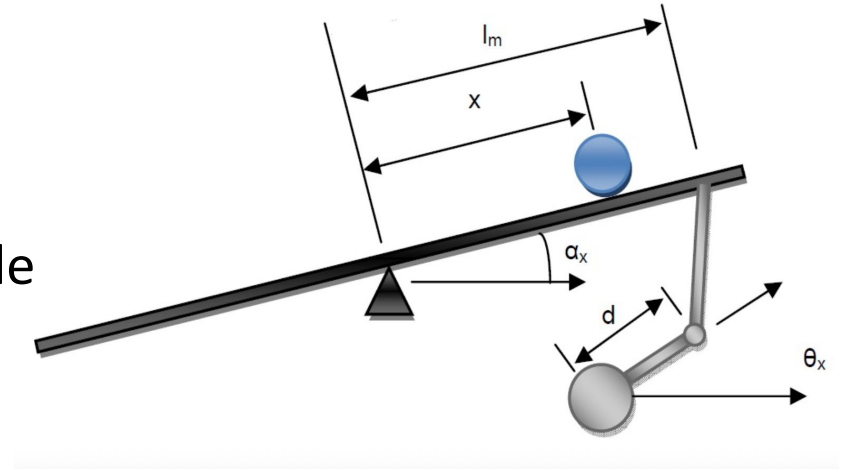
System Overview

- **Ball dynamics:**

Position $\rightarrow$ Velocity $\rightarrow$ Acceleration

- Acceleration depends on plate angle

- System is dynamic and nonlinear
(servo angle vs plate angle)



State Space Equations

States:

x_1 = Position ($\dot{x}_1 = x_2$)

x_2 = Velocity ($\dot{x}_2 = kga \cdot \theta$)

Input:

u = Plate angle (θ)

Output:

$y = x_1$ (Position)

2nd order State Observer

observer estimates system states from-

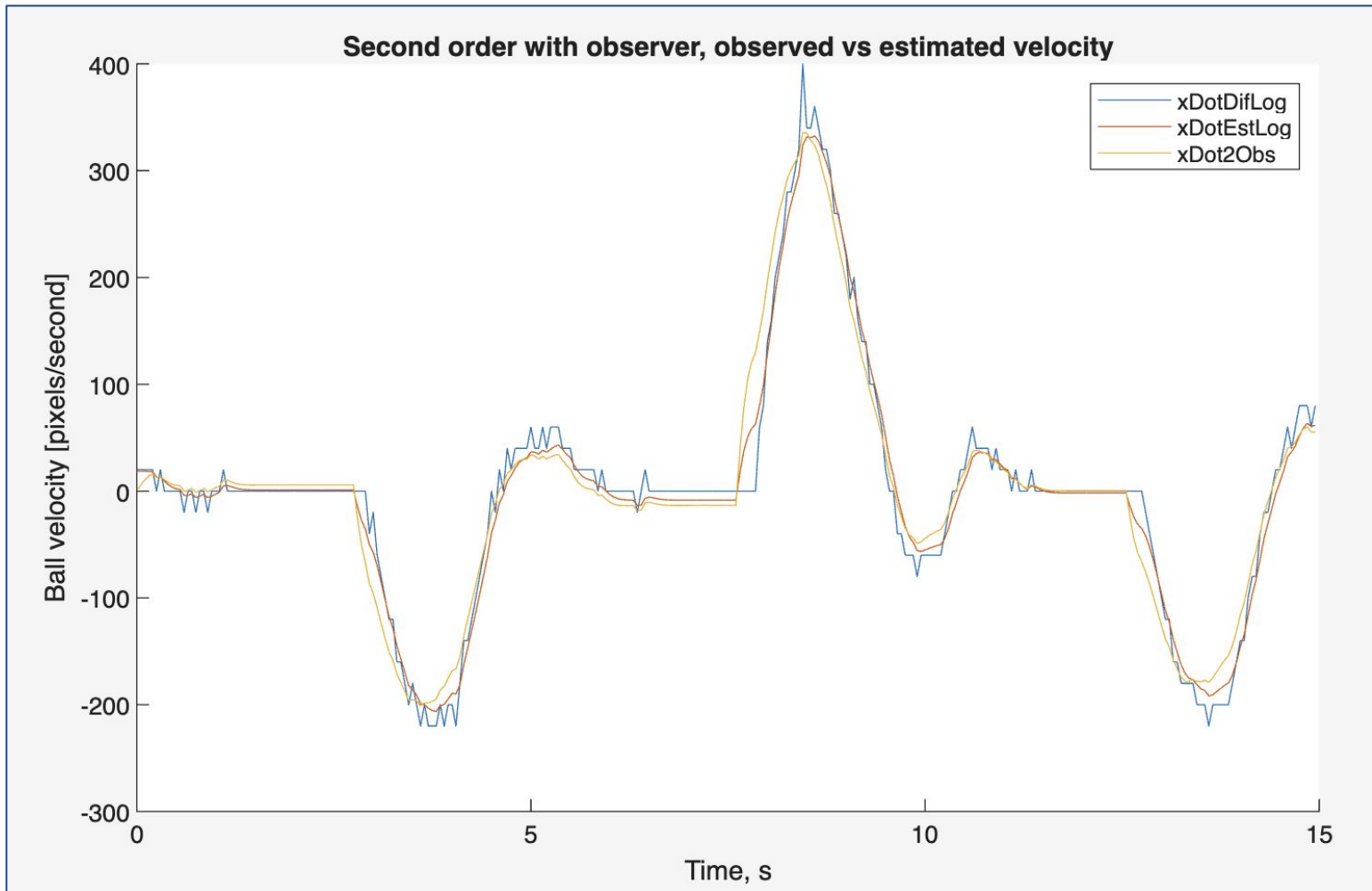
- System model
- Measured output

$$\hat{x}(k+1) = F\hat{x}^k + Guk + L(yk - \hat{y}^k)$$

Where:

- $\hat{x}$ = estimated state
- L = observer gain
- $(y - \hat{y})$ = innovation

2nd Order observer velocity plot



Limitations of 2nd order Observer

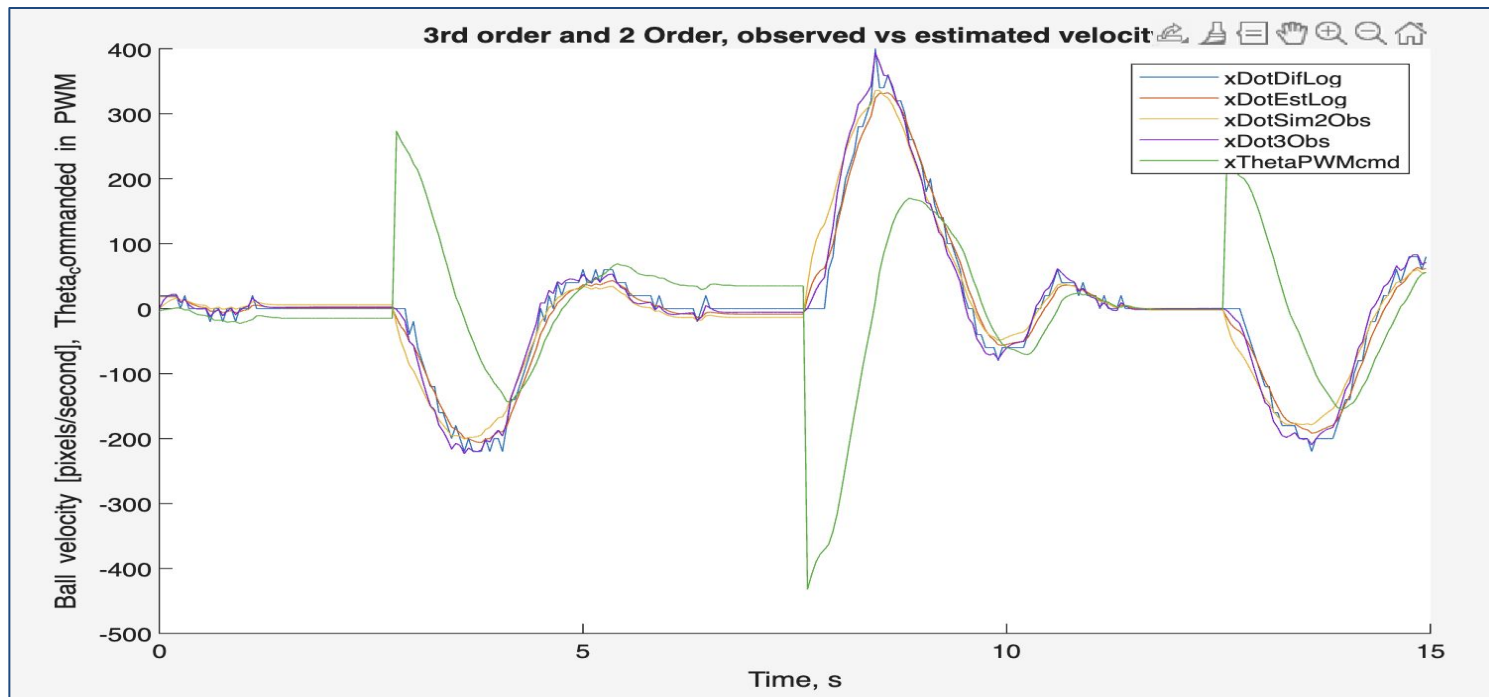
- Assumes perfect actuator
- Ignores servo dynamics
- Slower response in real system

3rd order Observer Model

States:

- x_1 =position
- x_2 =velocity
- x_3 =servo angle

servo time constant $a_s = 1/\tau$



From Luenberger to Kalman

- Same Observer Structure:

$$\hat{x}_{k+1} = A\hat{x}_k + Bu_k + L(y_k - C\hat{x}_k)$$

- Key Limitation:
 - Gain L chosen manually
 - No explicit Noise handling

Feature	Luenberger	Kalman
Gain	Fixed	Adaptive
Noise	Ignored	Modeled
Design	Pole placement	Optimal

Why do we need Kalman Filter?

- Real Systems:

$$x_{k+1} = Ax_k + Bu_k + w_k$$

$$y_k = Cx_k + v_k$$

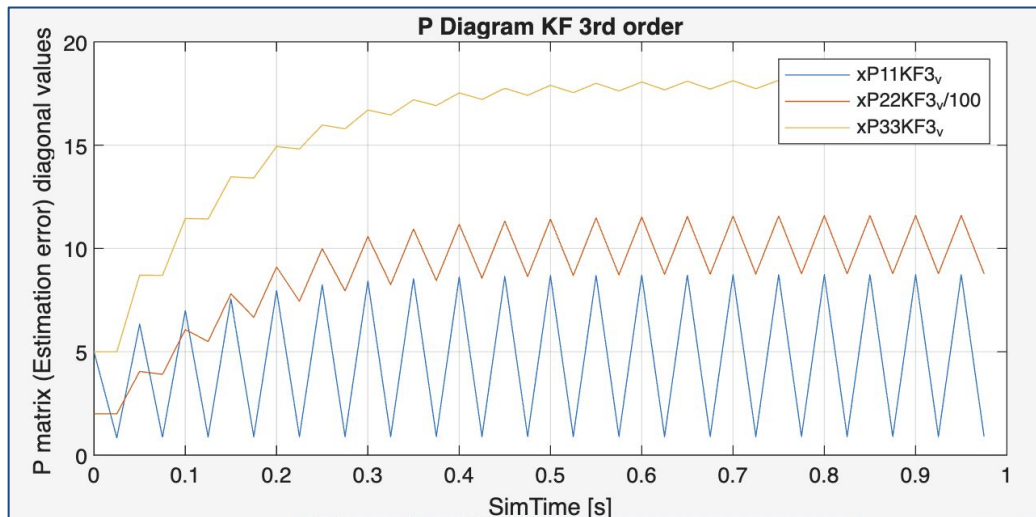
- w_k : Process noise (Q)
- v_k : Measurement noise (R)

- Key assumptions:

- Linear system
- Gaussian noise (mean free)
- Known covariances Q,R

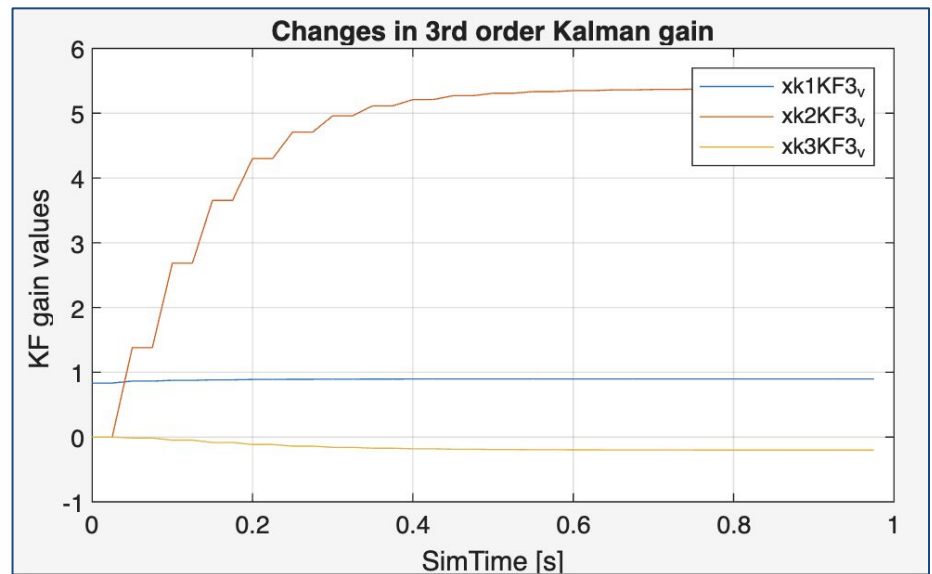
Kalman Filter Tuning

- Kalman gain K:
 - Balances Model vs Measurement
- If
 - Higher $Q \rightarrow$ trust measurement more
 - Higher $R \rightarrow$ trust model more
- P: uncertainty
 - Gets stable, independent of initial value

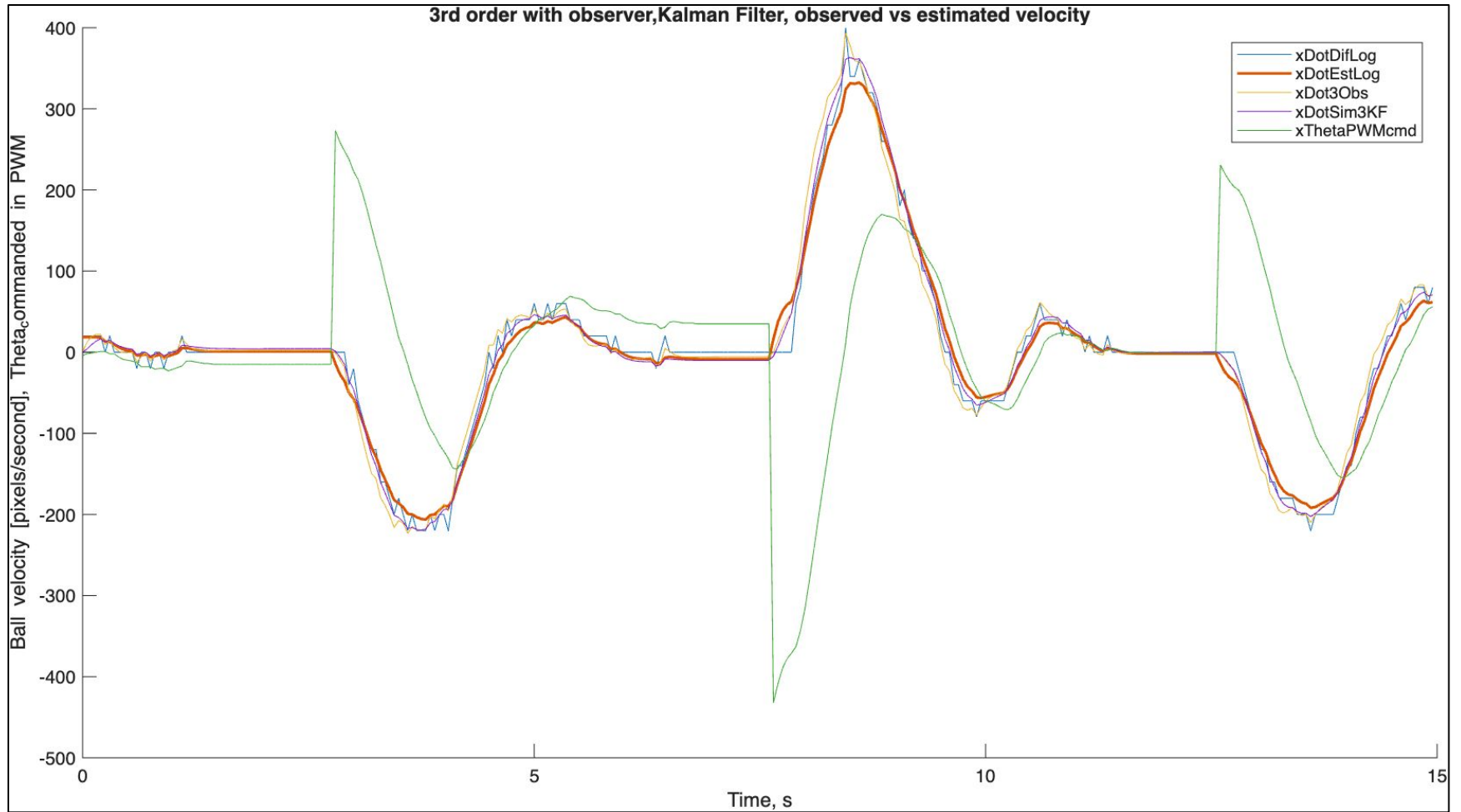


Results of Kalman Filter

- More accurate estimation as compared to observer model
- Smoother output
- Robust to noise
- Gain computed and updated as function of P, R, Q
- After few iterations
 - Gain converges to constant
 - System becomes stable



Plot of Kalman Filter 3rd order



Limitation of Kalman Filter

- Assumes System behaves linearly

$$\alpha(t) = \frac{d}{l_m} \theta(t)$$

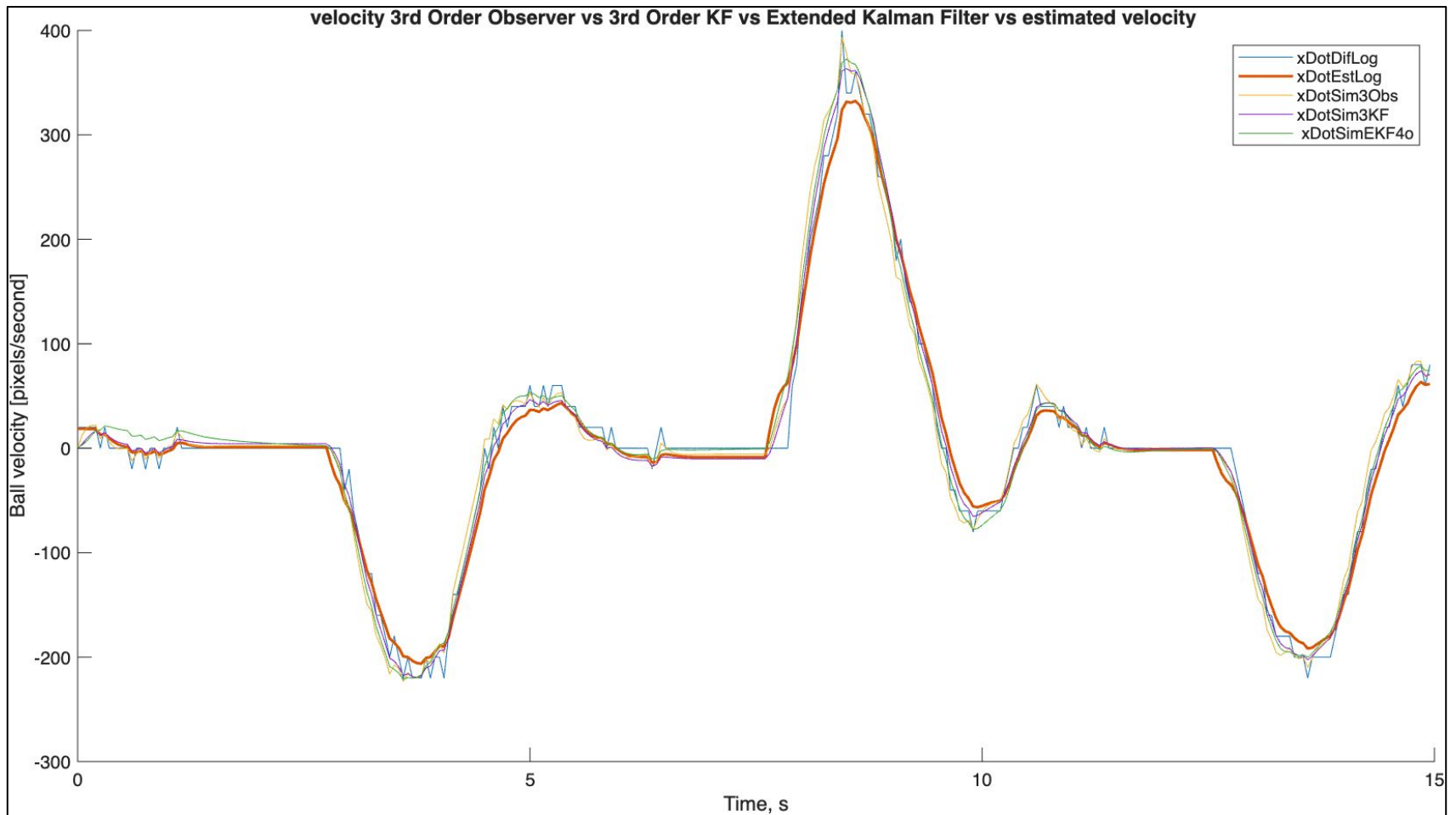
$$\alpha(t) = \frac{d}{l_m} \sin(\theta(t))$$

- Introduces error,
 - 5% at 30 deg
 - 12 % at 50 deg servo angle

Extended Kalman Filter

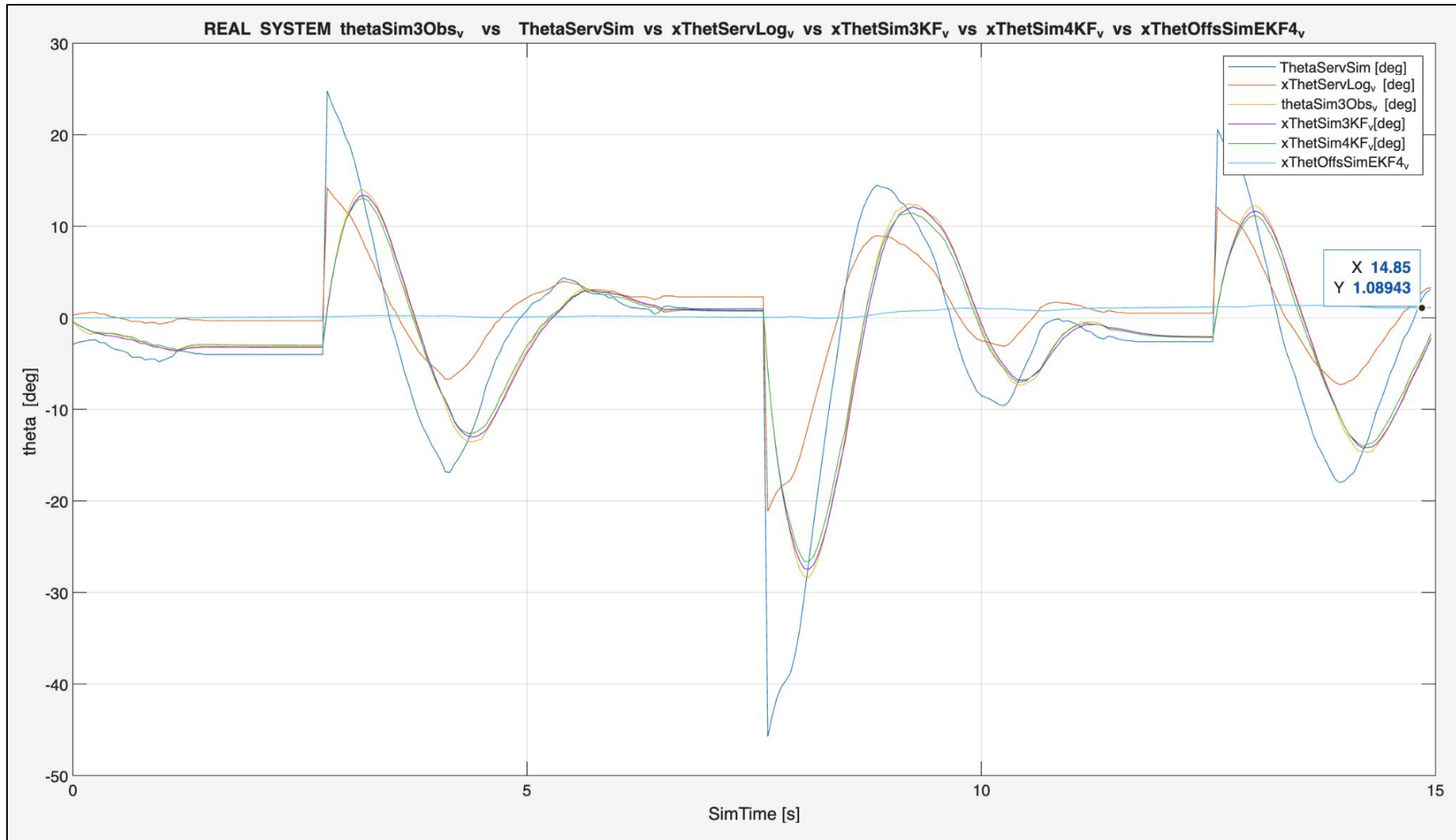
- Features:
 - Handles non linearity of System
 - Linearize using Jacobian matrix
 - Adds 4th state θ_{offset} as unknown parameter
 - F & G matrix calculated at each step
 - Apply Kalman framework
- Limitations:
 - Linearization error
 - Can diverge

Plot of Extended Kalman Filter



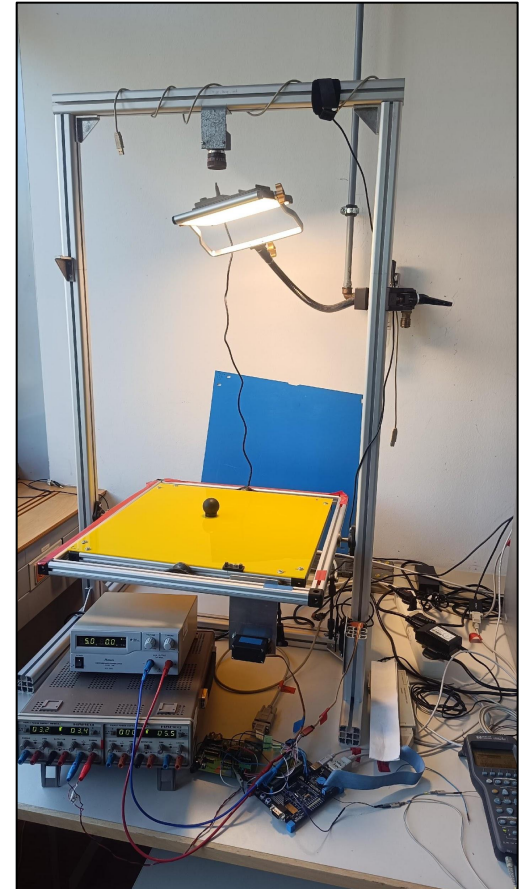
Plot of EKF offset Estimation

For PWM_XOFFS_BIASof 10 us



Implementation on Hardware

- Used same library as in Matlab
- adapted embedded Code
- STM32 integration
- Testing
- Optimizing
- additional Features



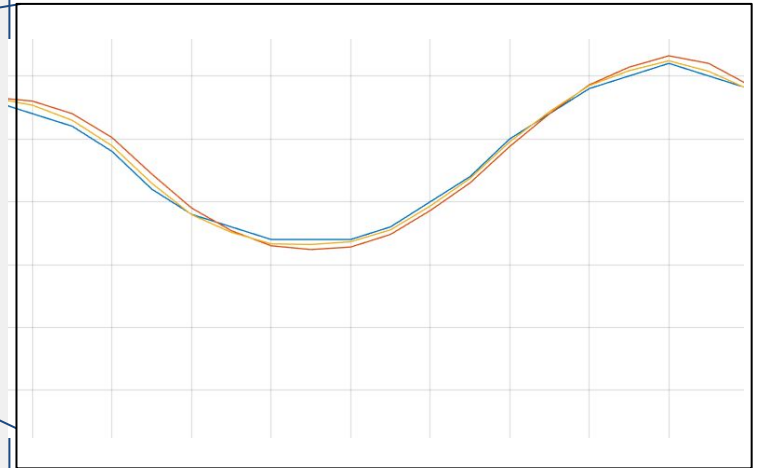
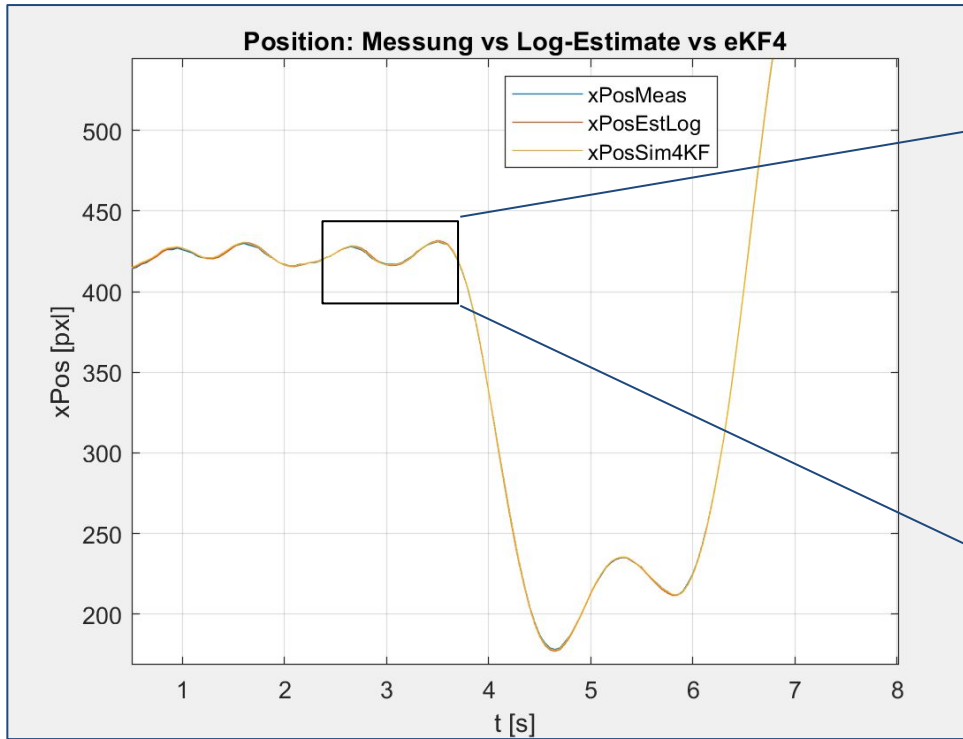
General Overview

- What to do:
 - get overview of the existing code
 - adapt the Ballplate-library from Matlab to μ Vision step by step and test it
 - watch out for units
 - no constant initialization of matrices
 - recalculate F and G every loop for x and y
 - don't use same P, G, KG for x and y
 - Thetaservo offset estimation printed every 100th loop

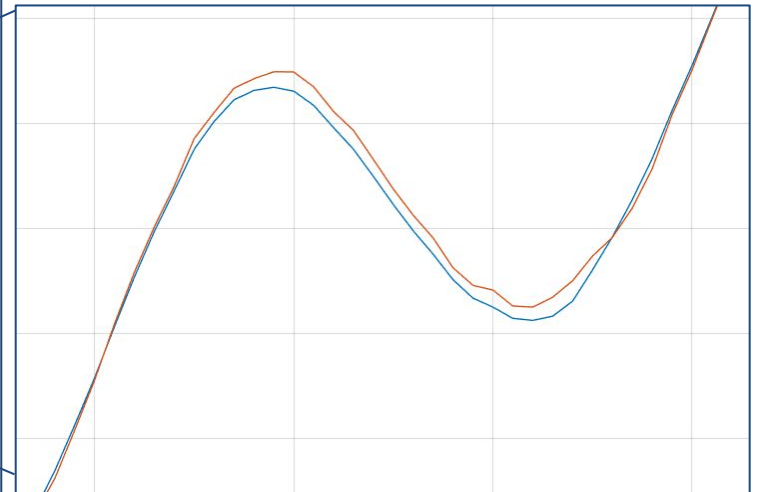
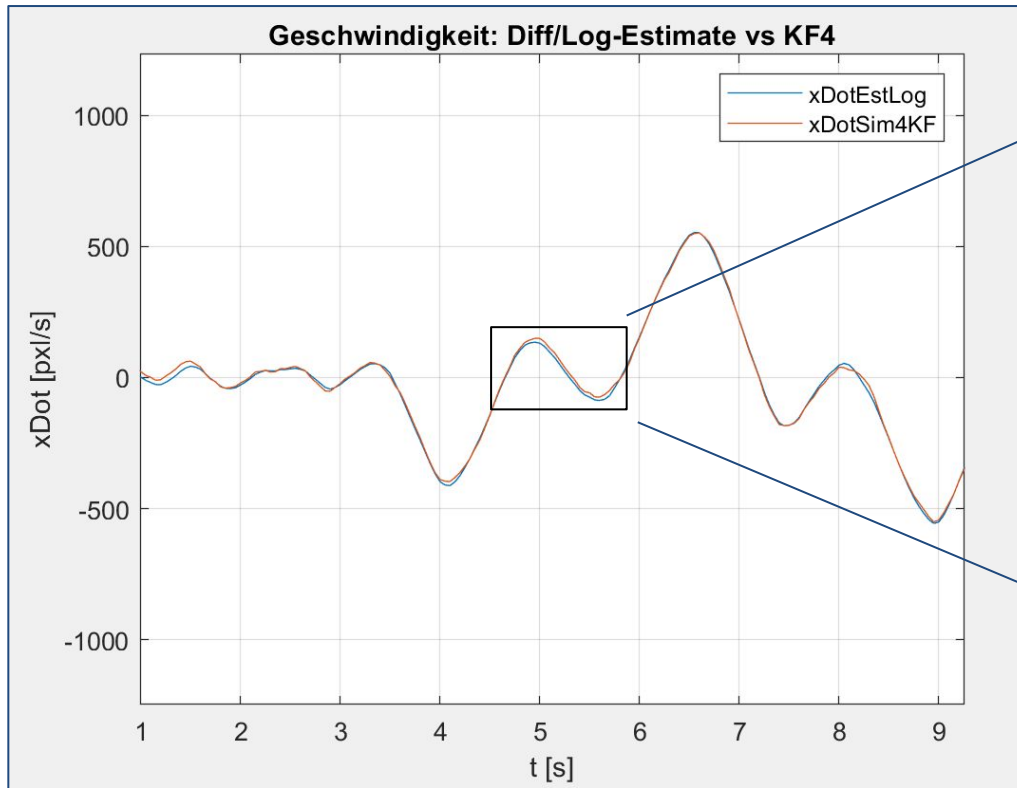
Problems along the Implementation

- unrecognized stack overflows and hardfaults
- lose cables and IC's
- defective PC Powersupply
- understanding the complexity and connection between different files and variables
- system behaves not exactly as the simulation

Implementation Results

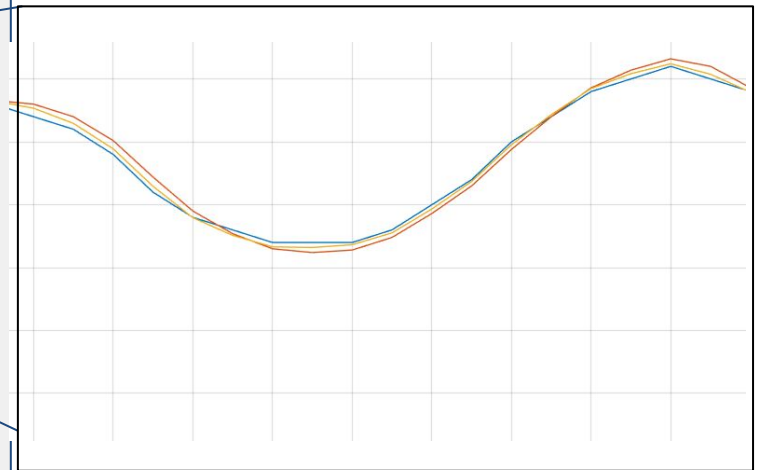
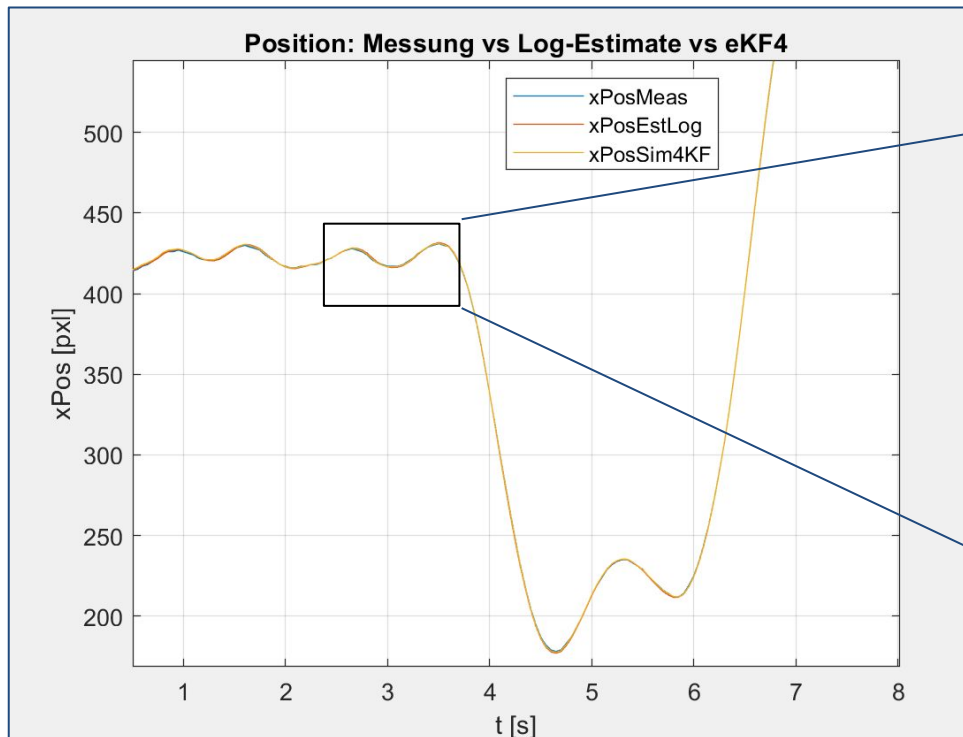


Implementation Results



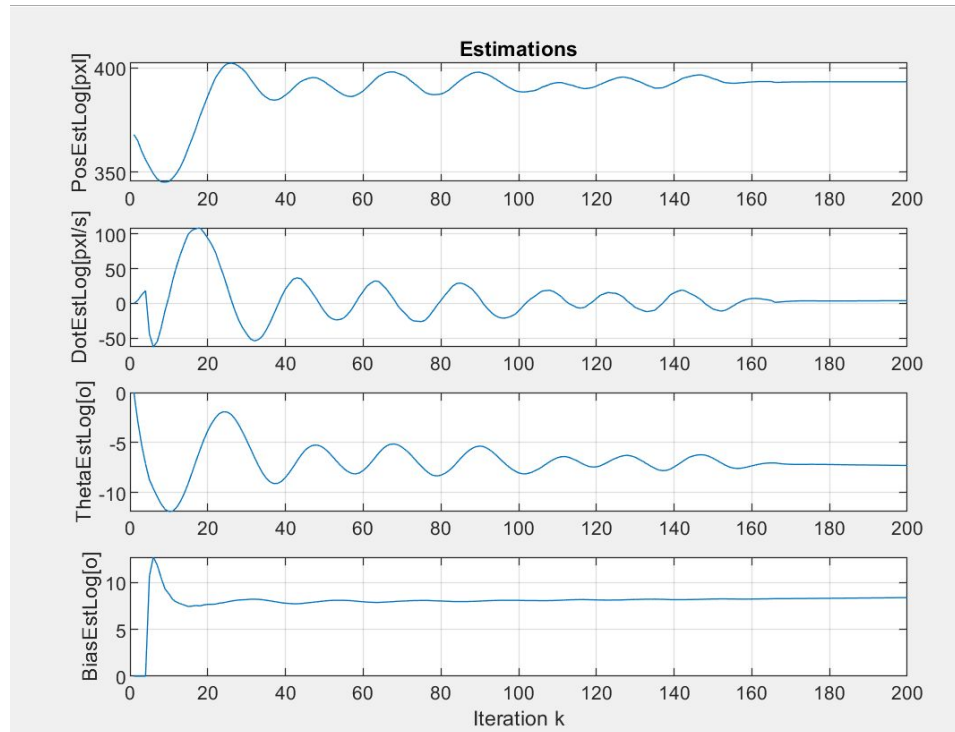
Implementation Results

- Faster adoption to Thetaoffsetbias -> slightly worse overall behavior (unstable)
- but for an big bias adapting very fast and the ball falls not from the plate



Implementation Results

- Faster adoption to Thetaoffsetbias -> slightly worse overall behavior (unstable)
- but for an big bias adapting very fast and the ball falls not from the plate



Conclusion and outlook

- Implemented Observer, KF & EKF in Matlab
- Implemented EKF4th order in an Embedded system
- Learned theory + practical
- Applied theory to real system
- steady state gets estimated correct and improves stability
- As the theory stated -> good controlling even with one sensor

Possible Improvements:

- More fine tuning on Parameters
- Tuning the state feedback controller
- PC-Powersupply for both 3.3V and 5V
- longer servo-arm to get faster without clipping